Authentication and Authorization: Design Document

1. Purpose and requirements

This document describes a prototype built to show two security tactics from Bass, Clements, and Kazman, *Software Architecture in Practice*, 4th ed. (SAiP), Chapter 11: **Authenticate Actors** and **Authorize Actors** (p. 174). Both belong to the "Resist Attacks" branch of the security tactics tree. Authenticate Actors makes sure that a caller is who it claims to be. Authorize Actors makes sure that an authenticated caller uses only the services it is entitled to.

The prototype is a small **clinic portal**: an HTTP API with nine services, such as reading a patient's record, recording vitals, writing a prescription, and reading the audit log. Five demo users of four types (ADMIN, DOCTOR, NURSE, PATIENT) call these services with HTTP Basic credentials. A clinic was chosen because it needs both kinds of authorization the textbook names: per role (a nurse may record vitals but not prescribe) and per actor (a patient may read their own record but not another patient's). The data is fictional and held in memory.

Every request crosses one Spring Security filter chain. The chain first authenticates the caller against bcrypt-hashed passwords, then authorizes the request against a fixed access table, and denies anything the table does not list. Failed logins and denied requests are recorded in an audit log (Audit, p. 176), so the two tactics leave evidence that can be inspected.

The security quality attribute is the subject of the whole document. Section 2 states it as two concrete scenarios, Sections 3 to 6 show the architecture that meets them, Section 7 rates the tactics we did not implement, Section 8 records the design decisions and their trade-offs, and Section 9 shows the evidence that the scenarios' response measures are met.

Requirements checklist

#	Assignment requirement	Where it is satisfied
R1	Use JAAS or the Spring Framework for the security tactics	Spring Security 7 with Spring Boot 4; JAAS is not used. README "Libraries and frameworks"; decision D1 in Section 8
R2	More than one user	Five seeded users (`DemoUsers`). README "Demo users"
R3	More than one user type	Four roles: ADMIN, DOCTOR, NURSE, PATIENT (`Role`). README "Demo users"; Section 4
R4	Different privileges for each user type	The access table and access matrix. Section 4; README "Services" and "Access matrix"
R5	Services identified by URLs	Nine method-and-URL services in `AccessRules`. README "Services (URL table)"; Section 4
R6	Implement the Authenticate Actors tactic	HTTP Basic, `DemoUserDetailsService`, bcrypt. Section 3 "Identify Actors and Authenticate Actors"; Section 6 login diagrams
R7	Implement the Authorize Actors tactic	URL rules generated from `AccessRules`, role hierarchy, `OwnRecordGuard`, default deny. Section 3 "Authorize Actors and Limit Access"; Section 6 request diagrams
R8	Runnable source code and an executable	Maven project; `mvn package` builds `target/authentication-and-authorization.jar`. README "Build and run"
R9	README with guidance to run the program	README "Prerequisites", "Build and run", "Demo"

#	Assignment requirement	Where it is satisfied
R10	README listing the libraries and frameworks used	README "Libraries and frameworks"
R11	Design document on the security quality attribute, covering both tactics	This document: scenarios (Section 2), tactics (Section 3), privilege matrix (Section 4), security view (Section 5), sequence diagrams (Section 6), tactics questionnaire (Section 7), decisions (Section 8), verification (Section 9)

2. Security scenarios

SAiP (Ch. 3 and Ch. 11) states a quality attribute requirement as a scenario with six parts: source, stimulus, artifact, environment, response, and response measure. We wrote one scenario per tactic. Each response measure is something the tests and the demo script can check, and Section 9 reports the result.

Scenario S1: Authenticate Actors

Part	Value
Source	An unidentified external human (or a script acting for one), outside the clinic's staff and patients, or a known user who mistypes a password
Stimulus	Requests a protected service, for example `GET /api/patients/1/record`, with no credentials, a wrong password, or a username that does not exist
Artifact	The clinic portal's HTTP API: the Spring Security filter chain in front of every controller
Environment	Normal operation, on a trusted local network; the server has just started or has been running for any time (no session state is kept)

Part	Value
Response	The request is refused with `401 Unauthorized` and the challenge `WWW-Authenticate: Basic realm="clinic"`. No controller runs and no protected data is disclosed. A wrong password and an unknown user receive identical responses, so the response does not reveal which usernames exist. A rejected username and password are recorded in the audit log as `AUTHN_FAILURE`; an anonymous request to a protected URL is recorded as `AUTHZ_DENIED` for `anonymous`
Response measure	100% of such requests are refused with 401; 0 bytes of protected data are returned; every attempt that presents a readable username and password produces an audit entry

Scenario S2: Authorize Actors

The layout follows the security scenario in Table 19.1 of SAiP.

Part	Value
Source	An authenticated internal actor: the PATIENT `bob` (patient 2), who holds valid credentials
Stimulus	Asks for another patient's data: `GET /api/patients/1/record` (Alice's record)
Artifact	The authorization rules: the URL rules generated from `AccessRules` and the per-actor check `OwnRecordGuard`
Environment	Normal operation, on a trusted local network

Part	Value
Response	The URL rule admits the request (a PATIENT may call the service), and the own-record check refuses it before the record is read. The caller receives `403 Forbidden` with a JSON body naming only the status, error and path. The denial is recorded in the audit log as `AUTHZ_DENIED`
Response measure	0 unauthorized grants across all 72 user-by-service cells of the access matrix (Section 4); a patient never learns whether another patient id exists (403 comes before any 404)

The same response applies to the per-role case: a NURSE posting a prescription, or any non-admin reading the audit log, is refused with 403 by the URL rule itself.

The trusted-network assumption

Both scenarios assume a **trusted local network**, and that assumption substitutes for transport encryption (TLS), which the prototype does not provide. HTTP Basic sends the username and password on every request, only Base64-encoded. Base64 is an encoding, not encryption: anyone who can read the traffic can recover the password. Over plain HTTP the credentials are therefore exposed on every request, not only at login. The prototype is safe to run on a developer laptop or a closed lab network; it is not safe to expose to an untrusted network as it stands. Section 7 rates this risk (Encrypt Data: not supported, risk High) and Section 8 records the decision.

3. Tactics and where they live

All the security in the portal lives in one place: a single Spring Security filter chain, declared in `SecurityConfig`, that every HTTP request crosses before any controller runs. This is the Intercepting Filter pattern that SAiP describes (p. 194). The controllers contain no security code of their own, apart from one `@PreAuthorize` annotation on each of the two patient read services, and they cannot be reached except through the chain. A new controller therefore cannot skip authentication or authorization by accident: if nobody adds a row for it to the access table, the chain's last rule denies it.

The filters that matter here run in this order for each request:

1. `BasicAuthenticationFilter` reads the `Authorization` header and authenticates the caller (Identify Actors, Authenticate Actors).
2. `ExceptionTranslationFilter` turns any later security exception into a response: 401 through `ApiAuthenticationEntryPoint` for an anonymous caller, 403 through `ApiAccessDeniedHandler` for an authenticated one.
3. `AuthorizationFilter` applies the URL rules generated from `AccessRules`, including the final default deny (Authorize Actors per role, Limit Access).
4. Method security (`@PreAuthorize`) runs `OwnRecordGuard` just before the two patient read methods (Authorize Actors per actor).

Spring Security publishes an event at each decision point, and `SecurityEventListener` turns those events into audit entries (Audit). Nothing in the request path calls the audit log directly.

An intermediary has a cost (SAiP, p. 179): every request pays for each filter it passes. The largest part of that cost here is not the filters themselves but the bcrypt comparison that stateless HTTP Basic repeats on every authenticated request, about 60 to 80 ms on a development laptop against a few milliseconds for an anonymous call. We accept that cost in exchange for a single, auditable place where security decisions are made (Section 8, decisions D3 and D5).

Identify Actors and Authenticate Actors

Identify Actors. The textbook describes this tactic as identifying "the source of any external input to the system", and notes that "users are typically identified through user IDs" (Bass, Clements, and Kazman, *Software Architecture in Practice*, 4th ed., p. 174). In the clinic portal the actor is identified by the username sent in the HTTP Basic `Authorization` header on every request (`admin`, `dr.chen`, `nurse.patel`, `alice`, `bob`). Nothing else identifies a caller: there is no session cookie and no token. A request without the header is treated as anonymous.

Authenticate Actors. The textbook defines authentication as "ensuring that an actor is actually who or what it purports to be", with passwords as the first example of a means to do it (p. 174). We authenticate with a username and password over HTTP Basic, following the Spring Security reference for Basic authentication (https://docs.spring.io/spring-security/reference/servlet/authentication/passwords/basic.html). The work is done by these classes, in the order a request meets them:

1. `BasicAuthenticationFilter` (Spring Security) is the filter in the chain that reads the `Authorization: Basic ...` header, decodes the username and password, and asks the authentication manager to check them. A request without the header passes through it unauthenticated.
2. `ProviderManager` (Spring Security) is the authentication manager. It hands the credentials to its provider and publishes an `AuthenticationSuccessEvent` when the provider accepts them, or a subclass of `AbstractAuthenticationFailureEvent` when it rejects them.

These events are how the audit listener learns about each attempt; nothing in our code calls the audit log directly. There is one gap: a malformed `Authorization: Basic` header (not Base64, or no `username:password` inside) is rejected by Spring's `BasicAuthenticationConverter` before `ProviderManager` runs, so the caller gets a 401 but no failure event is published and the attempt never reaches the audit log; a header with a scheme other than Basic is simply ignored, and the caller is treated as anonymous. Only attempts that present a username and password that can be read are recorded.

3. `DaoAuthenticationProvider` (Spring Security) loads the user by name and compares the password.
4. `DemoUserDetailsService` (our `authn` package) is the user store. It looks the name up among the five seeded users and returns a new `ClinicUser`, which carries the user's role and, for a patient, the patient id that the own-record rule needs later. An unknown name throws `UsernameNotFoundException` and nothing else.
5. `DelegatingPasswordEncoder` (Spring Security, created by `PasswordEncoderFactories.createDelegati` in `SecurityConfig`) checks the password. Stored passwords are `{bcrypt}` hashes; the prefix tells the delegating encoder to use BCrypt (cost 10). No plaintext password appears in the application code.
6. `ApiAuthenticationEntryPoint` (our `authn` package) writes the response when authentication fails.

How failure is surfaced. Every authentication failure ends in `ApiAuthenticationEntryPoint`, which answers `401 Unauthorized` with the challenge header `WWW-Authenticate: Basic realm="clinic"` and the JSON body `{"status":401,"error":"Unauthorized","path":"<request URI>"}`. That covers a wrong password, an unknown user, a malformed `Authorization` header, and an anonymous caller on any URL except the public one. Bad credentials are refused even on the public URL, because the Basic filter runs before any URL rule. The body never says why authentication failed. A wrong password and an unknown user produce byte-identical responses: `DaoAuthenticationProvider` reports an unknown user as the same `BadCredentialsException` as a wrong password, and still runs a BCrypt comparison against a dummy hash, so the response time does not give the difference away either. This is Spring's default behaviour; we did not configure it, so `AuthenticationTest` checks it (identical status, headers, body, and failure event for `nurse.patel:wrong` and `mallory:x`) rather than assuming it.

Every request is authenticated again. The filter chain is stateless: no HTTP session is created, so the server remembers nothing between requests. Each request carries the credentials and goes through the whole sequence above, including a BCrypt comparison. On a development laptop that costs roughly 70 to 80 ms per authenticated request, against about 5 ms for an anonymous call to the public URL. We accept that cost to keep the server free of session state. It also means that "authenticated" in the audit log means "this request was authenticated", not "this user logged in once". The sequence diagrams in

Section 6 show the success and failure paths message by message.

Authorize Actors and Limit Access

Authorize Actors, per role. Authorize Actors determines which services an authenticated actor may use (Bass, Clements, and Kazman, *Software Architecture in Practice*, 4th ed., p. 174). `AuthorizationFilter` checks the request's HTTP method and URL before a controller runs. `AccessPolicy` generates these rules from `AccessRules.all()`. It matches `PermitAll`, `Authenticated`, and `AnyRole` requirements to Spring Security's corresponding checks. Generating the rules keeps the runtime policy tied to the frozen access table instead of maintaining a second copy. `SecurityConfig` supplies the hierarchy ADMIN > DOCTOR > NURSE; each staff rule names its minimum role and inherits the higher roles. PATIENT is outside this hierarchy. For example, a nurse may record vitals but cannot write prescriptions.

Authorize Actors, per actor. A role alone does not establish whose record a patient may read. `PatientController.record` and `appointments` therefore carry `@PreAuthorize("@ownRecord.allows(authentication, #id)")`. Method security calls `OwnRecordGuard` before entering the controller body. A caller holding `ROLE_PATIENT` must have a `ClinicUser` principal with a patient ID equal to the requested ID; a missing ID or another principal type is refused. Staff callers pass this check because the URL rule has already restricted admission to patients or staff. This ordering prevents patients from probing whether another patient's ID exists: the guard refuses access before a repository lookup could return 404.

Limit Access. The final URL rule is `anyRequest().denyAll()`: a service absent from the table, including an unlisted HTTP method, is unavailable even to ADMIN. This implements Limit Access (p. 174), so adding a controller does not silently grant access. ERROR dispatches are permitted first so rendering an existing error does not cause another authorization denial; an ordinary request to an unlisted URL still meets default deny.

Both URL and method denials publish `AuthorizationDeniedEvent` through the publisher configured in `SecurityConfig`. The event object identifies either the request or the method invocation. For an authenticated caller, `ExceptionTranslationFilter` routes the exception to `ApiAccessDeniedHandler`, which uses `ApiError` to write status 403 and `{"status":403,"error":"Forbidden","path":"<request URI>"}`. Anonymous callers are sent to the 401 entry point instead. The audit listener records the event.

Audit

The `SecurityEventListener` applies the Audit tactic (SAiP 4th ed., p. 176). It records each authentication success and failure and each authorization denial in

the bounded, in-memory `AuditLog`, and writes the same event to the application log at INFO. Each entry includes the time, username, event kind, HTTP method and request path, plus a failure exception name or denial reason. With stateless HTTP Basic, authentication occurs independently on each request, so each authenticated request produces an `AUTHN_SUCCESS` entry rather than a one-time login entry.

4. Privilege matrix

Access table

The portal has nine services, each identified by an HTTP method and a URL pattern. The table is kept as data in `AccessRules`, and `AccessPolicy` generates the Spring Security URL rules from it in this order. "NURSE+" means NURSE or any role above it in the hierarchy ADMIN > DOCTOR > NURSE. PATIENT is outside the hierarchy. A "yes" in the last column means the per-actor own-record rule also applies: a PATIENT may call the service only for their own patient id.

#	Method	URL	Who may call it	Own-record rule
1	GET	`/api/public/info`	anyone, no login needed	
2	GET	`/api/me`	any logged-in user	
3	GET	`/api/patients/{id}/record`	PATIENT (own) or NURSE+	yes
4	GET	`/api/patients/{id}/appointments`	PATIENT (own) or NURSE+	yes
5	GET	`/api/patients`	NURSE+	
6	POST	`/api/patients/{id}/vitals`	NURSE+	
7	POST	`/api/patients/{id}/prescriptions`	DOCTOR+	
8	GET	`/api/admin/users`	ADMIN	
9	GET	`/api/admin/audit`	ADMIN	
–	any	anything else	nobody (default deny)	

Access matrix

The expected status for each demo user against each service. Patient ids: 1 is alice, 2 is bob, 99 does not exist. "anon" is a request without credentials. The source of this matrix is `scripts/demo-matrix.txt`, which both `scripts/demo.sh` and `AccessMatrixTest` read, so the documented, tested and demonstrated matrices are the same file.

Service	anon	admin	dr.chen	nurse.patel	alice (P1)	bob (P2)
GET /api/public/info	200	200	200	200	200	200
GET /api/me	401	200	200	200	200	200
GET /api/patients/1/record	401	200	200	200	200	403
GET /api/patients/2/record	401	200	200	200	403	200
GET /api/patients/1/appointments	401	200	200	200	200	403
GET /api/patients/99/record	401	404	404	404	403	403
GET /api/patients	401	200	200	200	403	403
POST /api/patients/1/vitals	401	201	201	201	403	403
POST /api/patients/1/prescriptions	401	201	201	403	403	403
GET /api/admin/users	401	200	403	403	403	403
GET /api/admin/audit	401	200	403	403	403	403
GET /api/nothing (unmapped)	401	403	403	403	403	403

Bad credentials are refused before any rule is consulted: a wrong password (`nurse.patel:wrong`) or an unknown user (`mallory:x`) gets 401 on every URL, including the public one.

The status codes mean:

- **401**: not authenticated (no credentials, wrong password, or unknown user).
- **403**: authenticated, but this user may not call this service.
- **404**: a staff member asked for a patient id that does not exist.
- **200 / 201**: success (read / write).

Per-role and per-actor access control

SAiP describes Authorize Actors as controlling access by the actor's identity, either per actor or per class of actor, such as a role (p. 174). The matrix shows both.

Per role. Most cells are decided by the caller's role alone, at the URL rule. The `POST /api/patients/1/prescriptions` row is the clearest: dr.chen gets 201, nurse.patel gets 403, because DOCTOR+ admits DOCTOR and ADMIN (through the hierarchy) but not NURSE. The `GET /api/admin/*` rows show the same thing at the top of the hierarchy, and the `GET /api/patients` and `POST .../vitals` rows show that a PATIENT is never treated as staff. The `GET /api/nothing` row shows Limit Access: the URL is in no rule, so even admin gets 403.

Per actor. In the `GET /api/patients/{id}/record` and `.../appointments` rows, alice and bob hold the same role, yet their cells differ. The URL rule admits both as PATIENT; `OwnRecordGuard` then compares the requested id with the caller's own patient id. So alice gets 200 for patient 1 and 403 for patient 2, and bob the reverse. The `GET /api/patients/99/record` row shows why the per-actor check runs before the data is read: staff get 404 because the id does not exist, but both patients get 403, so a patient cannot use the difference between 403 and 404 to discover which patient ids exist.

5. Security view

This view shows the classes that carry out the security tactics and how they connect to the Spring Security filter chain. Classes from Spring Security are grey and drawn as one group, the chain. Our classes are grouped by package. The `clinic` package is left out because it holds only domain data and no security logic.

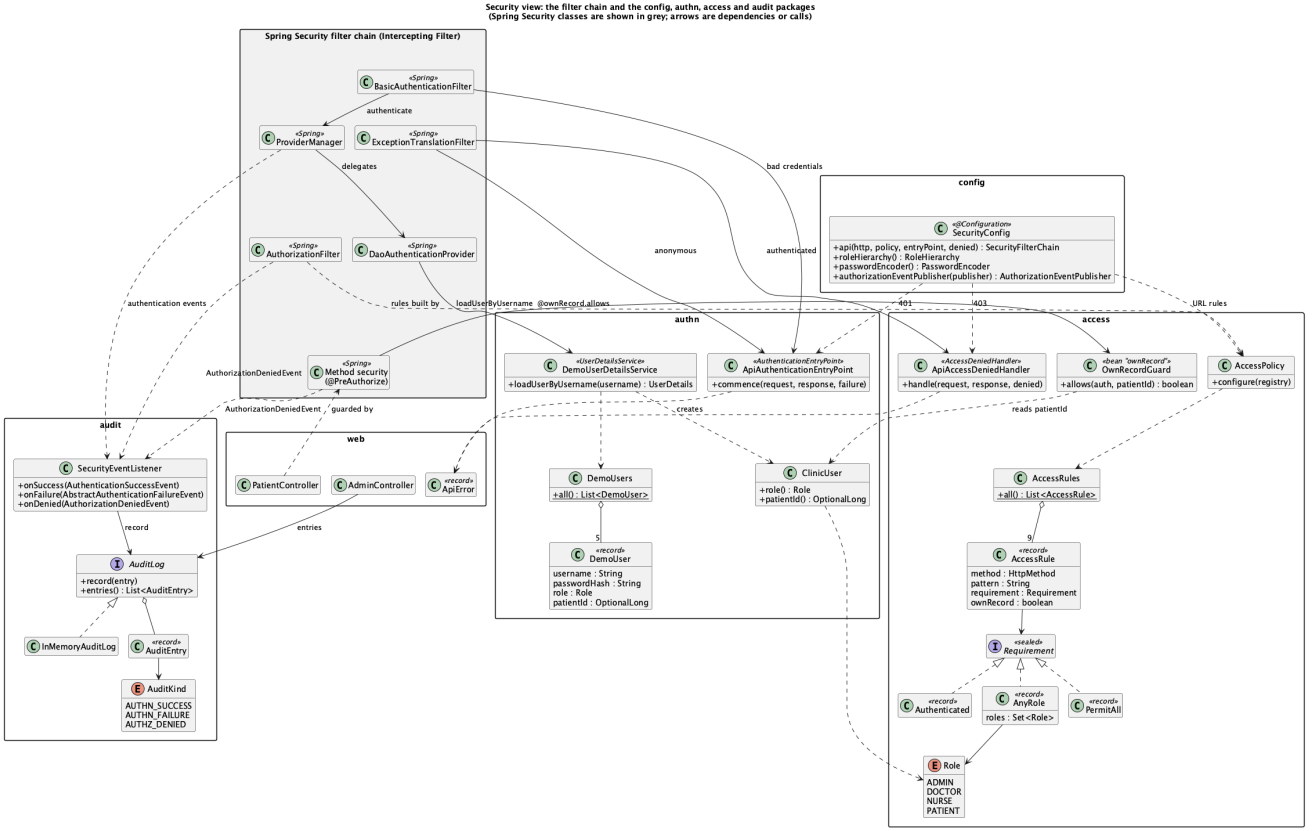


Figure 1: Security view

The PlantUML source is `uml/class/security-view.puml`, and a text outline of the diagram is in `uml/class/security-view.utxt`.

How to read it: a request enters at `BasicAuthenticationFilter` (top left) and meets the `authn` classes first. It then passes `AuthorizationFilter`, whose rules come from the `access` table, and for the two patient read services it also passes method security, which calls `OwnRecordGuard`. Every rejection leaves the chain through one of two classes, `ApiAuthenticationEntryPoint` (401) or `ApiAccessDeniedHandler` (403), and both write the same `ApiError` body. Every decision is published as an event and arrives at `SecurityEventListener` in `audit`. `SecurityConfig` connects all of this and contains no behaviour of its own.

Element catalog

`config` package. `SecurityConfig` is the only class that wires the chain. It builds the single `SecurityFilterChain` (HTTP Basic, stateless sessions, CSRF off, the 401 entry point and the 403 handler) and delegates the URL rules to `AccessPolicy`. It also declares the shared security beans: the role hierarchy `ROLE_ADMIN > ROLE_DOCTOR > ROLE_NURSE`, which both URL rules and method security honour; the delegating `PasswordEncoder`, which selects BCrypt from the `{bcrypt}` prefix of a stored hash; and the `AuthorizationEventPublisher`, without which Spring Security would not publish authorization denials and the audit log would miss them. `@EnableMethodSecurity` on the class turns on the `@PreAuthorize` checks. The class is part of the frozen contract.

`access` package (Authorize Actors, Limit Access). `AccessRules` holds the ordered service table as immutable `AccessRule` records. Each row supplies its HTTP method, path pattern, `Requirement` (`PermitAll`, `Authenticated`, or `AnyRole`), and own-record marker. `Role` names the four actor types; the staff hierarchy is supplied by `config.SecurityConfig`. `AccessPolicy` turns the table into URL authorization rules, permits ERROR dispatch first, and denies unlisted requests last. `OwnRecordGuard`, named `ownRecord`, is the per-actor check called by the two patient read methods' `@PreAuthorize` annotations: patients must match their principal's patient ID, while staff already admitted by URL authorization may read any patient. `ApiAccessDeniedHandler` writes a consistent JSON 403 response using `web.ApiError` for an authenticated caller who fails either check. The configured Spring authorization event publisher reports both kinds of denial for the audit listener.

`authn` package (Identify Actors, Authenticate Actors). These classes are the application's side of HTTP Basic authentication; the filter, the authentication manager and the provider that call them come from Spring Security. `DemoUser` is a record for one seeded user: username, `{bcrypt}` password hash, role, and, for a patient, the patient id. `DemoUsers` holds the five seeded users; plaintext passwords never appear in the application code. `DemoUserDetailsService` is the `UserDetailsService` that Spring's `DaoAuthenticationProvider` consults

on every request that carries Basic credentials: it returns a new `ClinicUser` for a known name and throws only `UsernameNotFoundException` for an unknown one, so that Spring can report an unknown user exactly like a wrong password. `ClinicUser` extends Spring's `User` and is the authenticated principal; besides the single `ROLE_*` authority it carries the role and patient id, which the own-record rule reads during authorization without a second lookup. `ApiAuthenticationEntryPoint` is the `AuthenticationEntryPoint` installed on both HTTP Basic and exception handling; it writes every 401 with the challenge `WWW-Authenticate: Basic realm="clinic"` and the JSON body `{"status":401,"error":"Unauthorized","path":...}` (built by the shared `web.ApiError` record), and never reveals why authentication failed.

`audit` package (Audit). `SecurityEventListener` subscribes to three Spring Security events: `AuthenticationSuccessEvent`, `AbstractAuthenticationFailureEvent` and `AuthorizationDeniedEvent`. It turns each into an `AuditEntry` that holds the time, an `AuditKind` (`AUTHN_SUCCESS`, `AUTHN_FAILURE`, `AUTHZ_DENIED`), the username (`anonymous` for a caller without credentials), the HTTP method and path, and a detail: the exception name for a failed login, or the required roles, `denyAll`, `authenticated`, or the guarded method name for a denial. The listener appends the entry to the `AuditLog` and writes it to the application log at INFO. `InMemoryAuditLog` is the only `AuditLog`. It is synchronized for concurrent requests and keeps the newest 1000 entries, so memory stays bounded. Because the listener reacts to events, no controller or filter of ours calls the log directly.

`web` package. The controllers are the services of the access table and contain no security checks of their own, with one exception. `PatientController.record` and `appointments` carry `@PreAuthorize("@ownRecord.allows(authentication, #id)")`, the hook for the per-actor rule. `AdminController` exposes the user list (usernames and roles only, never password hashes) and the audit log. `MeController` reports the caller's own username, role and patient id. `PublicController` is the one service that needs no login. `ApiError` is the shared `{"status","error","path"}` body that both the 401 and the 403 responses use, so a rejection never reveals more than its status and path.

`clinic` package (not drawn). `ClinicRepository` holds the patients, records, appointments, vitals and prescriptions in memory. It throws `PatientNotFoundException` (mapped to 404) for an unknown id. It is reached only after authorization has succeeded, which is why a patient sees 403, never 404, for another patient's id.

6. Sequence diagrams

Login success and login failure

Both diagrams follow one request to `GET /api/me`, the service that only asks for an authenticated caller. Under stateless HTTP Basic there is no separate login

step: each request carries the username and password, so "login" here means the authentication that happens at the start of every request. The PlantUML sources and text renderings are next to the images in `uml/sequence/`.

Login success

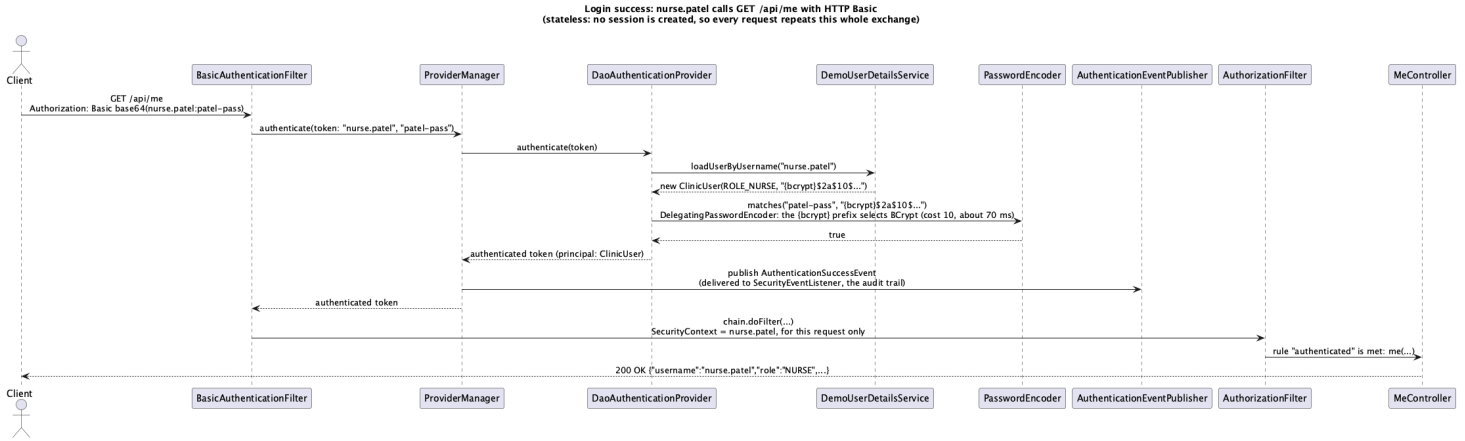


Figure 2: Login success

1. The client sends `GET /api/me` with `Authorization: Basic` and the Base64 of `nurse.patel:patel-pass`. `BasicAuthenticationFilter` decodes the header into a username and password token. This is where the actor is identified, by username.
2. The filter passes the token to `ProviderManager`, which hands it to `DaoAuthenticationProvider`.
3. The provider asks `DemoUserDetailsService` for `nurse.patel`. The service builds a new `ClinicUser` holding the stored `{bcrypt}` hash, the role `NURSE`, and no patient id. It is a new object on every call because Spring erases the password from the principal after a successful check.
4. The provider calls `PasswordEncoder.matches` with the presented password and the stored hash. The delegating encoder sees the `{bcrypt}` prefix and runs BCrypt, which takes about 70 to 80 ms. This comparison is the authentication decision: `true` means the caller is who it claims to be.
5. The provider returns an authenticated token whose principal is the `ClinicUser`. `ProviderManager` publishes an `AuthenticationSuccessEvent`, which the audit listener (`SecurityEventListener`) receives.
6. `BasicAuthenticationFilter` stores the authentication in the security context for this request only and continues down the chain. `AuthorizationFilter` finds that the rule for `/api/me` requires only an authenticated caller, so the request reaches `MeController`, which answers `200 OK`.

Nothing is kept afterwards. The next request from the same client repeats all six steps.

Login failure

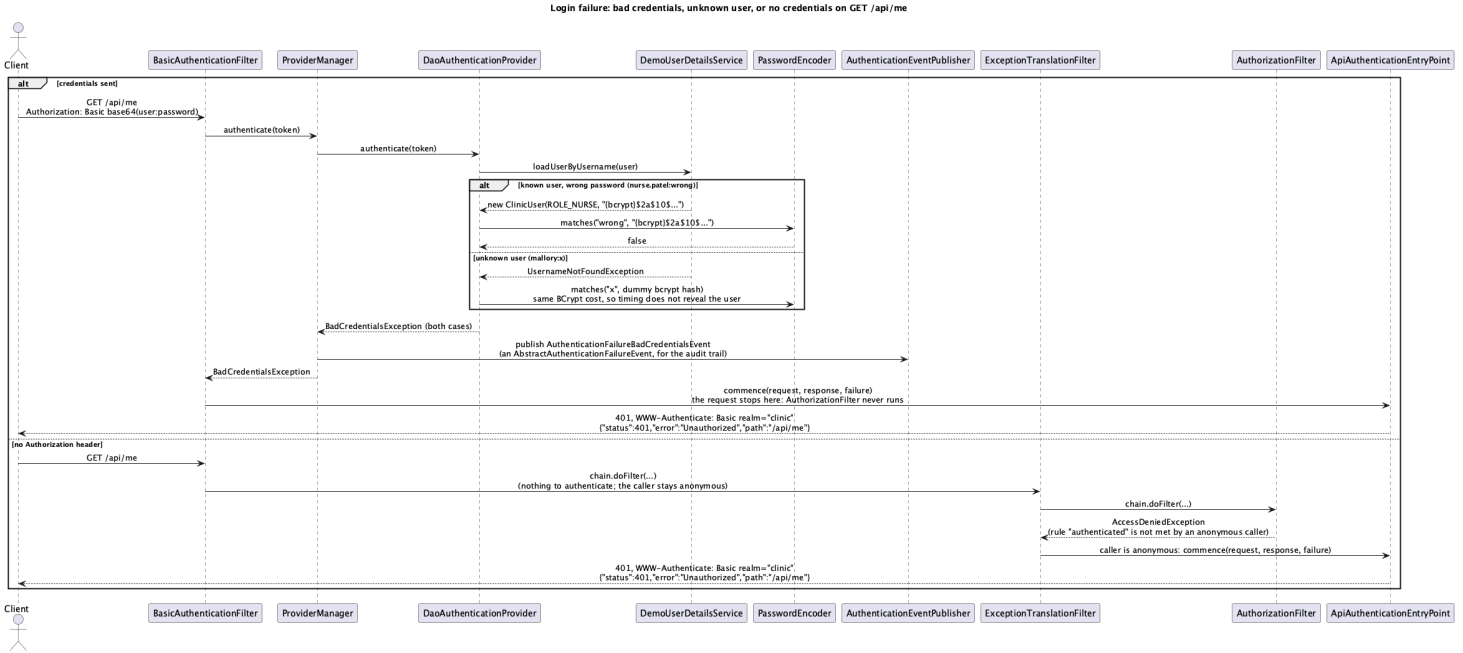


Figure 3: Login failure

The diagram has two outer cases.

Credentials sent but wrong. The first four messages are the same as in the success case. The difference is inside `DaoAuthenticationProvider`, shown as an inner choice:

- *Known user, wrong password* (`nurse.patel:wrong`): the user service returns a `ClinicUser`, and `PasswordEncoder.matches` returns `false`.
- *Unknown user* (`mallory:x`): the user service throws `UsernameNotFoundException`. The provider still runs a BCrypt comparison against a dummy hash, so an unknown name takes about as long as a wrong password.

Both cases leave the provider as the same `BadCredentialsException`; this is where the failure is decided, and where the two cases stop being distinguishable. `ProviderManager` publishes an `AuthenticationFailureBadCredentialsEvent` (a subclass of `AbstractAuthenticationFailureEvent`) for the audit listener and rethrows. `BasicAuthenticationFilter` catches the exception and calls `ApiAuthenticationEntryPoint.commence`, which answers `401` with `WWW-Authenticate: Basic realm="clinic"` and `{"status":401,"error":"Unauthorized","path":"/api/me"}`. The request never reaches `AuthorizationFilter` or a controller, which is why bad credentials get a 401 even on the public URL.

No credentials. With no `Authorization` header, `BasicAuthenticationFilter` has nothing to check and lets the request continue as anonymous. `ExceptionTranslationFilter` passes it on to `AuthorizationFilter`, which refuses it because an anonymous caller does not meet the rule "authenticated", and throws `AccessDeniedException`. `ExceptionTranslationFilter` catches it, sees that the caller is anonymous, and sends it to the same entry point instead of the 403 handler. The caller gets the same 401 and challenge as in the first case. An anonymous request to a URL that nobody may call, such as `/api/nothing`, also gets 401 this way: the caller is asked who they are before being told what they may not do.

Authorized request and forbidden request

These diagrams follow authenticated requests through the two authorization checks: the URL's role requirement in `AuthorizationFilter`, followed by the patient ownership check through `@PreAuthorize` and `OwnRecordGuard`. The first request passes both checks; the second passes the role check but fails ownership and receives 403. Each diagram's PlantUML source and UTXT rendering are stored beside its PNG under `uml/sequence/`.

Authorized request

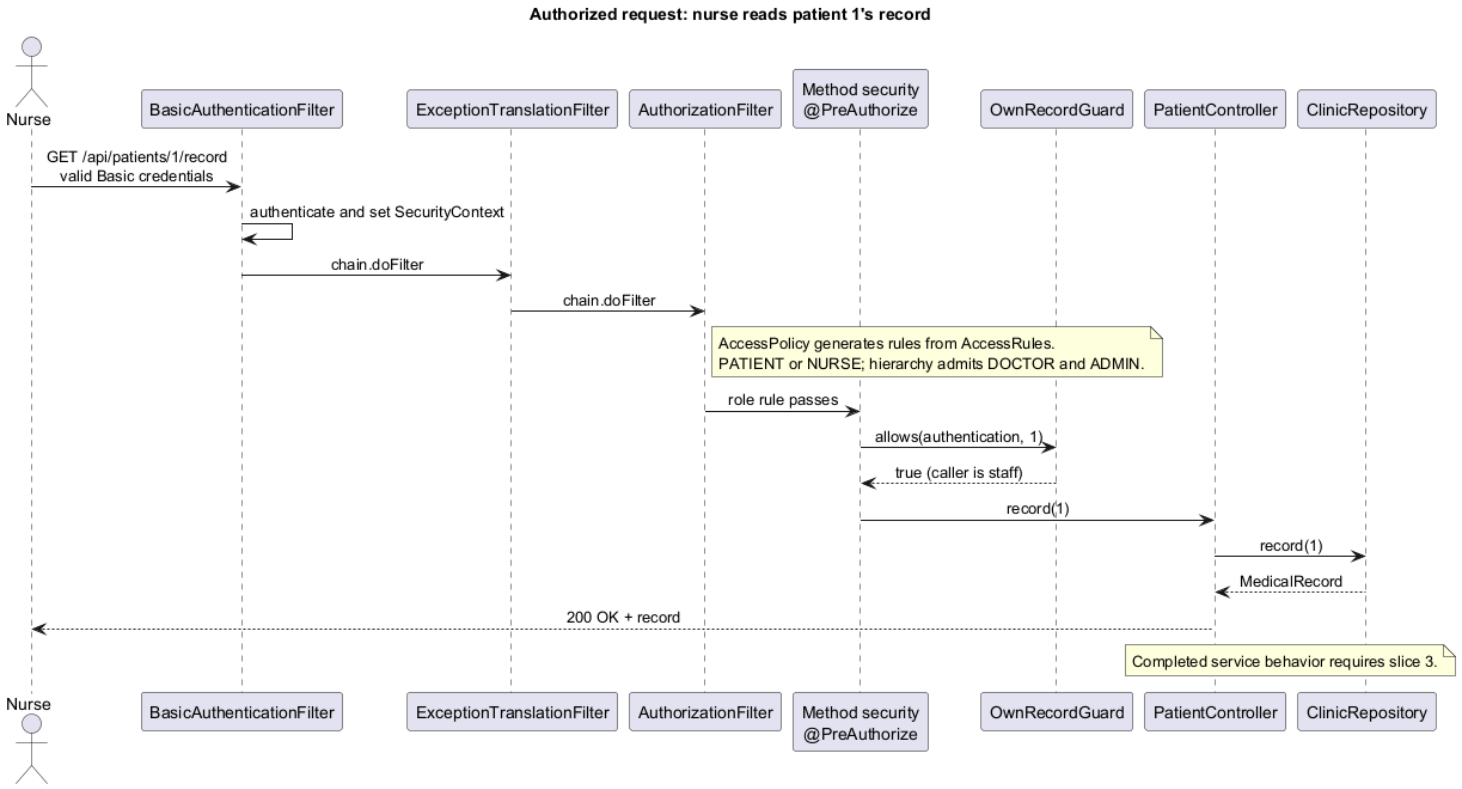


Figure 4: Authorized request

A nurse requests `GET /api/patients/1/record` with valid Basic credentials. Authentication establishes the `ClinicUser` before authorization. `AuthorizationFilter` checks the rule generated by `AccessPolicy` from `AccessRules`: PATIENT or NURSE is required, with DOCTOR and ADMIN also admitted through the staff hierarchy. Method security then calls `OwnRecordGuard`. The nurse does not hold `ROLE_PATIENT`, so the guard allows the call. Only then does the controller read the repository and return the record with 200.

Forbidden request

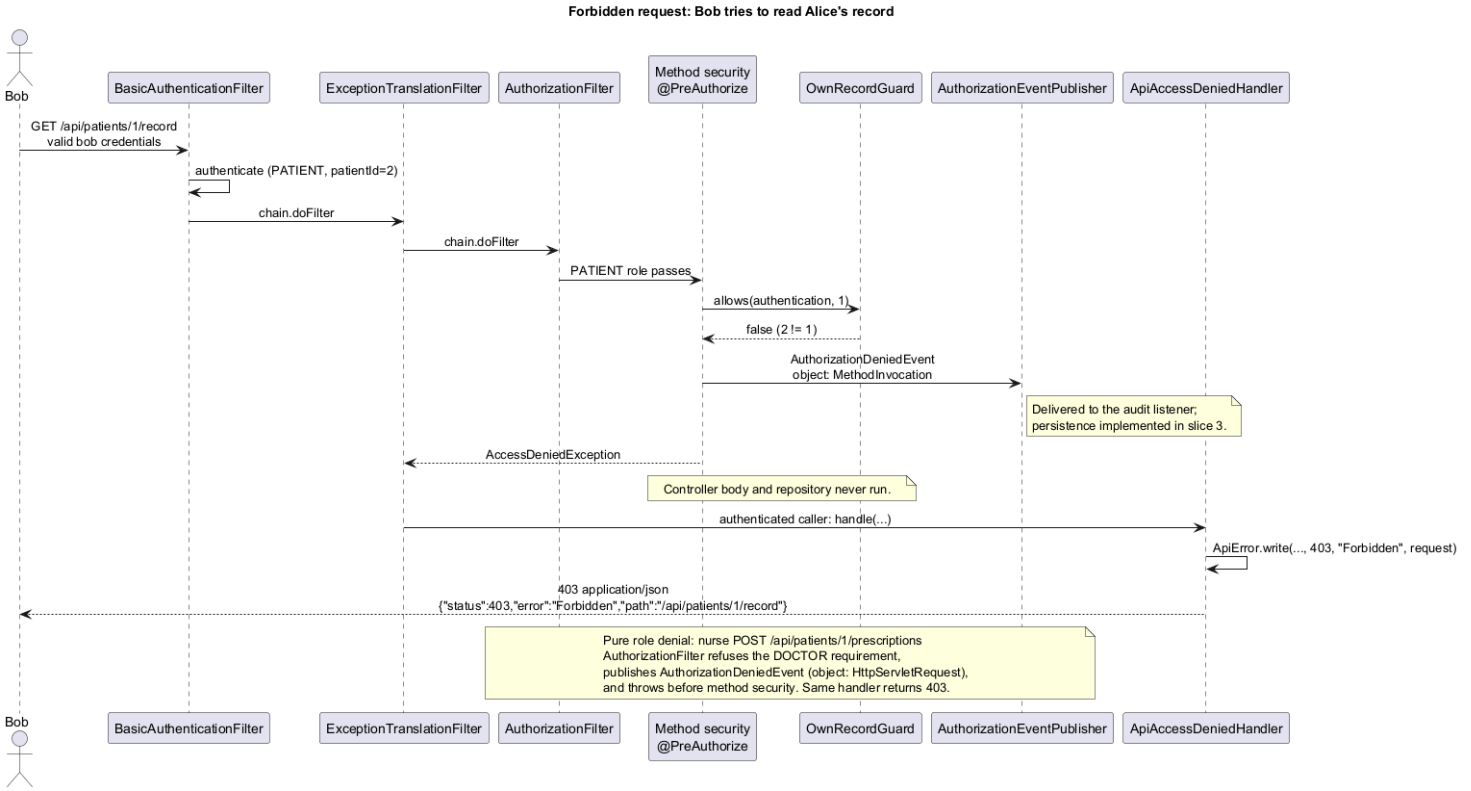


Figure 5: Forbidden request

Bob requests Alice's record at `/api/patients/1/record`. His PATIENT role passes URL authorization, but method security calls `OwnRecordGuard` and finds that his principal's patient ID is 2. The check returns false, publishes an `AuthorizationDeniedEvent` whose object is a `MethodInvocation`, and throws an access-denied exception. The controller body and repository are never called. `ExceptionTranslationFilter` recognizes an authenticated caller and invokes `ApiAccessDeniedHandler`, which returns JSON with status 403 and the requested path.

The side note shows a pure role denial: a nurse posting a prescription is stopped by `AuthorizationFilter`, which publishes a denial event whose object is the HTTP request. Method security is never reached, and the same handler returns 403. An anonymous caller is sent to the 401 entry point instead. The audit listener receives both kinds of denial event through the publisher configured in `SecurityConfig`.

7. Tactics questionnaire

The table follows the tactics-based questionnaire for security in SAiP (Table 11.2). It has one row per tactic in Chapter 11's security tactics tree, plus rows for the authentication features a reviewer would expect to see discussed (multi-factor authentication, session timeout, token-based authentication). "Supported" means the prototype implements the tactic. "Risk" rates what the gap would cost if this design were deployed beyond its trusted-network assumption: H (high), M (medium), L (low). For a supported tactic, the risk is the risk that remains.

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Detect Attacks	**Detect Intrusion**: does the system compare request patterns with known attack signatures?	N	M	None. Failed logins and denials are recorded (Audit) but not analysed	Out of scope for a two-tactic prototype. The audit log is the raw material an intrusion detector would read

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Detect Attacks	**Detect Service Denial**: does the system detect floods of requests?	N	M	None	No rate limiting. Each authenticated request costs a bcrypt check (about 70 ms), so a flood of requests with credentials is cheap to send and costly to serve. Acceptable on a trusted network
Detect Attacks	**Verify Message Integrity**: are messages protected against tampering (checksums, MACs)?	N	M	None	Integrity in transit would come from TLS, which is not used (see Encrypt Data)

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Detect Attacks	**Detect Message Delay**: are replayed or delayed messages detected?	N	L	None	HTTP Basic requests carry no nonce or timestamp, so a captured request can be replayed. Covered by the trusted-network assumption
Resist Attacks	**Identify Actors**: are the sources of external input identified?	Y	L	The username in the HTTP Basic header on every request; read by `BasicAuthenticationFilter` without (Section 3)	No session or token: the caller is identified again on each request. A request without the header is anonymous

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Resist Attacks	**Authenticate Actors**: are actors' identities verified?	Y	M	HTTP Basic, `DaoAuthenticationProvider` with `DemoUserDetailsService`, `{bcrypt}` hashes checked by the delegating `PasswordEncoder` (`SecurityConfig`, `authn`)	Passwords are stored only as BCrypt hashes (cost 10). A wrong password and an unknown user give identical 401 responses and similar timing. Risk stays M because the credentials cross the network in reversible form (Encrypt Data)
Resist Attacks	**Multi-factor authentication**: is more than one factor required?	N	M	None	One factor (password). A second factor would not fit stateless HTTP Basic without a separate login exchange

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Resist Attacks	**Token-based authentication** (for example OAuth 2.0 / JWT bearer tokens)	N	M	None; HTTP Basic sends the password itself on every request (decision D2)	Tokens would limit how often the password crosses the network and allow expiry, but would need a token issuer. Basic keeps the authentication path short and visible for the course
Resist Attacks	**Authorize Actors**: is access controlled per actor or per role?	Y	L	Per role: URL rules generated from `AccessRules` by `AccessPolicy`, with the role hierarchy from `SecurityConfig`. Per actor: `OwnRecordGuard` through `@PreAuthorize` (`access`)	The table is data, so the rules, the tests and the README come from one source. All 72 cells of the access matrix are checked (Section 9)

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Resist Attacks	**Limit Access**: is access to resources restricted beyond what is needed?	Y	L	`anyRequest().denyAll()` as the last URL rule (`AccessPolicy`); stateless, no session; no browser client	A URI not listed in the table is denied to every user, ADMIN included. A new controller is unreachable until it is given a row
Resist Attacks	**Limit Exposure**: are the number of access points and the data exposed kept small?	Y	L	Nine services only; error bodies hold only status, error and path (`ApiError`); `/api/admin/users` returns no hashes; a patient gets 403 before any 404	The 403-before-404 order means a patient cannot discover which patient ids exists

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Resist Attacks	**Encrypt Data**: are data and credentials encrypted in transit and at rest?	N	H	None. The server listens on plain HTTP; data is held in memory unencrypted	HTTP Basic over plain HTTP sends Base64-encoded, reversible credentials on **every** request. The only mitigation is the assumption that the network is trusted (Section 2). A deployment would need TLS before anything else
Resist Attacks	**Separate Entities**: are sensitive parts isolated from the rest (separate processes, networks)?	N	L	None. One process, one in-memory store	Not needed for a prototype; the security logic is at least separated into its own packages and a single filter chain

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Resist Attacks	**Validate Input**: is input checked for well-formedness and range?	Partly	M	Path ids are bound to `long` by Spring MVC, so a non-numeric id is rejected; JSON bodies are bound to typed records	No range or business validation of vitals or prescriptions. Not a focus of the two tactics
Resist Attacks	**Change Credential Settings**: are default credentials changed and credentials rotated?	N	M	Seeded users in `DemoUsers` with published demo passwords	The passwords are public on purpose so each user type can be tried (README). A real system would keep users outside the code and force a change on first login (p. 175)

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
React to Attacks	**Revoke Access**: can an actor's access be withdrawn while the system runs?	N	M	None. Users and roles are fixed in code	Because nothing is cached between requests, removing a user from the store would take effect on the next request; there is just no service to do it
React to Attacks	**Restrict Login**: are repeated failed logins limited (lockout, back-off)?	N	M	None. Failed attempts are recorded as `AUTHN_FAILURE` but not counted	Without lockout, passwords can be guessed online. bcrypt slows each guess to about 70 ms, which helps but does not stop it

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
React to Attacks	**Re-authentication / session timeout**: are idle sessions ended?	N/A	L	No sessions exist (`SessionCreationPolicy.STATELESS`)	Every request is authenticated again, so there is no session to expire. The cost is a bcrypt check per request (decision D3)
React to Attacks	**Inform Actors**: are operators or users told about an attack?	N	L	None. An administrator can read `GET /api/admin/audit`, but nothing is pushed	Notification would be built on the audit events

Tactic group	Tactic question	Supported? (Y/N)	Risk (H/M/L)	Design Decisions and Location	Rationale and Assumptions
Recover from Attacks	**Audit**: are security-relevant actions recorded?	Y	M	`SecurityEventListener` records `AUTHN_SUCCESS`, `AUTHN_FAILURE` and `AUTHZ_DENIED` into `InMemoryAuditLog` and the application log (`audit`)	Bounded to the newest 1000 entries, and lost on restart. A malformed Basic header is refused before an event is published, so it is not recorded (Section 3)
Recover from Attacks	**Nonrepudiation**: can an actor deny having made a request?	N	M	None	Audit entries are not signed and the password is shared knowledge in this demo, so an entry does not prove who acted

8. Design decisions

The table uses the layout of SAiP Table 22.4. Each row names the decision and where it lives in the code, then gives the rationale, the assumptions it depends on, and the alternative we discarded.

#	Design Decisions and Location	Rationale and Assumptions (Include Discarded Alternatives)
D1	**Spring Security (with Spring Boot) implements both tactics.** `pom.xml`; `SecurityConfig`	The assignment allows JAAS or Spring. Spring Security gives a filter chain in front of every HTTP request, URL-based authorization rules, method security, password encoders and security events, which match the tactics directly. *Discarded: JAAS.* JAAS provides pluggable login modules but has no model of HTTP URLs, so the URL-to-role rules would have to be written by hand. Its authorization side relied on the Java Security Manager, which JEP 486 permanently disabled in JDK 24.

#	Design Decisions and Location	Rationale and Assumptions (Include Discarded Alternatives)
D2	**HTTP Basic authentication on every request.** `SecurityConfig.api` (`httpBasic`); `ApiAuthenticationEntryPoint`	The clients are curl, the demo script and the tests, none of them browsers. Basic puts the whole authentication exchange in one header, so each request shows Identify Actors and Authenticate Actors end to end, and 401 versus 403 is easy to show. Assumes a trusted network (see the limitation below). *Discarded: form login,* which needs a login page, a session cookie and CSRF tokens. These suit a browser, add steps that are not part of either tactic, and would be unused by our clients.

#	Design Decisions and Location	Rationale and Assumptions (Include Discarded Alternatives)
D3	**Stateless sessions with CSRF protection off.** `SecurityConfig.api` (`SessionCreationPolicy.STATELESS`, `csrf.disable()`)	No session is created, so the server holds no login state and every request is authenticated again. CSRF exploits credentials a browser sends automatically (cookies). Our clients send credentials only when told to, so turning CSRF off is safe **only because there is no browser client**. A browser front end would make this decision wrong. *Discarded: server sessions with CSRF protection,* which would avoid repeating bcrypt on each request but add session fixation, timeout and CSRF concerns that the API does not otherwise have.

#	Design Decisions and Location	Rationale and Assumptions (Include Discarded Alternatives)
D4	**A role hierarchy ADMIN > DOCTOR > NURSE, with PATIENT outside it.** `SecurityConfig.roleHierarchy`; rules in `AccessRules` name only the lowest staff role	Each rule names its minimum role ("NURSE+"), and the hierarchy adds the roles above it. Method security honours the same bean. PATIENT is kept out so that a patient is never treated as staff. *Discarded: a flat `hasAnyRole("NURSE","DOCTOR","ADMIN")` on every rule,* which repeats the ordering in every rule. Adding a role or changing the order would then mean editing every rule, and one missed rule would silently give wrong access.
D5	**bcrypt password hashes through the delegating encoder.** `DemoUsers` (`{bcrypt}` hashes); `SecurityConfig.passwordEncoder`	Only hashes are stored. BCrypt is salted and deliberately slow (cost 10), which slows offline guessing if the store leaks. The `{id}` prefix lets a later change of algorithm coexist with old hashes. *Discarded: `{noop}` plaintext passwords,* which are simpler but store the secret itself and show nothing about credential storage.

#	Design Decisions and Location	Rationale and Assumptions (Include Discarded Alternatives)
D6	**A custom `UserDetailsService` that returns a `ClinicUser` principal.** `DemoUserDetailsService`, `ClinicUser`	The own-record rule needs the caller's patient id. Putting it on the principal lets `OwnRecordGuard` read it with no second lookup. The service builds a new principal on every call because Spring erases the password on the returned object. *Discarded: Spring's `InMemoryUserDetailsManager`,* which returns plain `User` objects that have only a name and authorities, so the patient id would need a second, separate mapping.
D7	**The access table is data, and the URL rules are generated from it, ending in default deny.** `AccessRules`, `AccessPolicy`	One source of truth: `AccessPolicy` builds the Spring rules from it, the tests assert against it, and `DocsConsistencyTest` checks the README against it, so the policy cannot drift from its documentation. The last rule, `anyRequest().denyAll()`, is Limit Access. *Discarded: hand-written `requestMatchers` chains in the configuration,* which are just as correct on the day they are written but give the tests and documentation nothing to check against.

#	Design Decisions and Location	Rationale and Assumptions (Include Discarded Alternatives)
D8	**The per-actor check runs in method security, before the controller body.** `@PreAuthorize("@ownRecordGuard.isOwn(authentication, #id)")` on `PatientController`; `OwnRecordGuard`	The check runs before the repository is read, so a patient gets 403 for any id other than their own, a missing id included, and cannot learn which ids exist. *Discarded: checking inside the controller after loading the record,* which would answer 404 for a missing id and 403 for an existing one, and so leak which ids exist. It would also spread security checks into domain code.
D9	**Audit through Spring Security events.** `SecurityConfig.authorizationEventPublisher`; `SecurityEventListener`; `InMemoryAuditLog`	The listener records every authentication success and failure; every authorization denial, without any controller or filter of ours calling the log. The log is in memory and bounded to the newest 1000 entries. *Discarded: calling the audit log from each controller,* which misses requests that never reach a controller, and those are exactly the rejected ones.

Trade-offs

bcrypt on every request. Under stateless HTTP Basic, the expensive part of authentication runs on every authenticated request, not once per login. In our demo run, authenticated requests took about 60 to 80 ms each, almost all of it BCrypt, against a few milliseconds for an anonymous call to the public URL. That is a deliberate trade of performance for security and simplicity: no session state, no session attacks, and a slow hash that also slows password guessing. A system with heavy traffic would exchange the password once for a short-lived

token or session, which brings back the state that D3 avoids.

The intermediary cost of the filter chain. SAiP notes that an intermediary adds processing to every message that passes through it (p. 179). Every request, including the public one, crosses the whole chain. Apart from bcrypt, the filters cost little. In return, all security decisions are made in one place that can be read, tested and audited, and a controller cannot be reached around it.

Hierarchy and generation add indirection. A reader has to look in three places (`AccessRules`, `AccessPolicy` and the hierarchy bean) to see why a request was admitted. The `AccessRules` javadoc, Section 4 and the README all show the resulting table, so this indirection does not hide the policy.

Limitation: HTTP Basic without TLS

HTTP Basic sends the username and password with every request, encoded in Base64, which anyone who sees the traffic can decode. The prototype serves plain HTTP, so on an untrusted network every request would expose the caller's password. This is the largest risk in the design (Section 7, Encrypt Data, risk High). The design is acceptable only under the trusted-network assumption of Section 2. Before any real deployment, the server would have to be put behind TLS. Ideally the password would also be exchanged for an expiring token, so that it crosses the network once rather than on every request.

9. Verification

The response measures of Section 2 are checked in two independent ways: by the JUnit tests that `mvn verify` runs on every build, and by `scripts/demo.sh`, which sends real HTTP requests to the packaged jar. Both read the access matrix from the same file, `scripts/demo-matrix.txt`, and CI runs both on every pull request.

Test summary (`mvn verify`)

Test class	Tests	What it covers
`authn.AuthenticationTest`	25	Scenario S1: a missing credential, a wrong password and an unknown user all get 401 with the `Basic` challenge and the JSON body on every protected URL; an unknown user is byte-for-byte identical to a wrong password in status, headers, body and failure event; anonymous requests to unmapped URLs and to writes are 401, not 403; a malformed or foreign `Authorization` header is 401, not 500; each request re-authenticates with no session; success and failure events are published
`authn.ApiAuthenticationEntryPointTest`	2	The 401 entry point on its own: the status, the `WWW-Authenticate: Basic` challenge and JSON content type, and a response that never reveals the failure reason

Test class	Tests	What it covers
`access.AuthorizationTest`	14	Scenario S2 denials: per-role URL denials with the 403 JSON body, default deny on an unmapped URL and on a wrong method even for admin, per-actor own-record denials, a patient blocked from recording vitals before method security runs, URL- versus method-level denial events, admin admitted through the role hierarchy, and the permitted error dispatch
`access.OwnRecordGuardTest`	7	The own-record guard as a unit: admin, doctor and nurse may read any patient; a patient only their own id; a non-`ClinicUser` principal and a patient with no id are refused; ids are compared by value
`access.ApiAccessDeniedHandlerTest`	4	The 403 handler: the status, the JSON body with the path escaped, the content length, no challenge header, and no leak of the denial reason

Test class	Tests	What it covers
`audit.AuditTest`	6	The three kinds of event and their detail; bare denials recorded as `authenticated` and as `denyAll` (including a listed path under an unlisted method); a forbidden request through the real chain logs a success then a denial; a crafted username with control characters and line separators cannot forge an extra log line; the entries list is an unmodifiable snapshot
`web.ServicesTest`	11	Each clinic service with the filters off: list patients, read a record and appointments, 201 echoing the stored vitals and prescription, 404 for an unknown patient id, `/api/me` carrying the patient id for a patient, null for staff and a fallback for a non-clinic principal, the admin user list without hashes, and the audit as an ISO-8601 JSON array
`web.ApiErrorTest`	3	`ApiError.write`: the exact body with JSON content type and length, the path escaped with the fields in order, and a clean body even when a stale response was already buffered

Test class	Tests	What it covers
`ContractSmokeTest`	8	The shared contract: the context loads with the policy as a plain component, the nine access rows in order, the five seeded users with bcrypt hashes matching their passwords, a fresh `ClinicUser` per lookup, an unknown user throwing `UsernameNotFoundException`, anonymous public info 200 through the real chain, the matrix file parsing to 80 rows, and the audit log keeping the newest 1000 entries oldest-first
`docs.DocsConsistencyTest`	1	The README's URL and user tables match the code, every section file exists with its heading, and no `TODO` marker remains anywhere in the README or the section files
`AccessMatrixTest`	82	Every one of the 80 rows of `scripts/demo-matrix.txt` through the real filter chain with real HTTP Basic credentials, plus two audit checks: a failed login is recorded as `AUTHN_FAILURE`, and a denial is recorded as `AUTHN_SUCCESS` then `AUTHZ_DENIED`

Totals: 164 tests, 0 failures, 0 errors, 0 skipped, `BUILD SUCCESS`.

Demo transcript (`scripts/demo.sh` against the packaged jar)

The jar was started with `java -jar target/authentication-and-authorization.jar --server.port=18731` and the demo was run with `scripts/demo.sh http://localhost:18731`. All 80 rows matched:

```
demo: http://localhost:18731
EXPECTED ACTUAL METHOD PATH                                USER                RESULT
200      200    GET    /api/public/info                    -                   OK
200      200    GET    /api/public/info                    admin               OK
200      200    GET    /api/public/info                    dr.chen             OK
200      200    GET    /api/public/info                    nurse.patel         OK
200      200    GET    /api/public/info                    alice               OK
200      200    GET    /api/public/info                    bob                 OK
401      401    GET    /api/me                             -                   OK
200      200    GET    /api/me                             admin               OK
200      200    GET    /api/me                             dr.chen             OK
200      200    GET    /api/me                             nurse.patel         OK
200      200    GET    /api/me                             alice               OK
200      200    GET    /api/me                             bob                 OK
401      401    GET    /api/patients/1/record              -                   OK
200      200    GET    /api/patients/1/record              admin               OK
200      200    GET    /api/patients/1/record              dr.chen             OK
200      200    GET    /api/patients/1/record              nurse.patel         OK
200      200    GET    /api/patients/1/record              alice               OK
403      403    GET    /api/patients/1/record              bob                 OK
401      401    GET    /api/patients/2/record              -                   OK
200      200    GET    /api/patients/2/record              admin               OK
200      200    GET    /api/patients/2/record              dr.chen             OK
200      200    GET    /api/patients/2/record              nurse.patel         OK
403      403    GET    /api/patients/2/record              alice               OK
200      200    GET    /api/patients/2/record              bob                 OK
401      401    GET    /api/patients/1/appointments        -                   OK
200      200    GET    /api/patients/1/appointments        admin               OK
200      200    GET    /api/patients/1/appointments        dr.chen             OK
200      200    GET    /api/patients/1/appointments        nurse.patel         OK
200      200    GET    /api/patients/1/appointments        alice               OK
403      403    GET    /api/patients/1/appointments        bob                 OK
401      401    GET    /api/patients/99/record             -                   OK
404      404    GET    /api/patients/99/record             admin               OK
404      404    GET    /api/patients/99/record             dr.chen             OK
404      404    GET    /api/patients/99/record             nurse.patel         OK
403      403    GET    /api/patients/99/record             alice               OK
403      403    GET    /api/patients/99/record             bob                 OK
401      401    GET    /api/patients                       -                   OK
200      200    GET    /api/patients                       admin               OK
```

```
200     200   GET   /api/patients                    dr.chen            OK
200     200   GET   /api/patients                    nurse.patel        OK
403     403   GET   /api/patients                    alice              OK
403     403   GET   /api/patients                    bob                OK
401     401   POST  /api/patients/1/vitals           -                  OK
201     201   POST  /api/patients/1/vitals           admin              OK
201     201   POST  /api/patients/1/vitals           dr.chen            OK
201     201   POST  /api/patients/1/vitals           nurse.patel        OK
403     403   POST  /api/patients/1/vitals           alice              OK
403     403   POST  /api/patients/1/vitals           bob                OK
401     401   POST  /api/patients/1/prescriptions    -                  OK
201     201   POST  /api/patients/1/prescriptions    admin              OK
201     201   POST  /api/patients/1/prescriptions    dr.chen            OK
403     403   POST  /api/patients/1/prescriptions    nurse.patel        OK
403     403   POST  /api/patients/1/prescriptions    alice              OK
403     403   POST  /api/patients/1/prescriptions    bob                OK
401     401   GET   /api/admin/users                 -                  OK
200     200   GET   /api/admin/users                 admin              OK
403     403   GET   /api/admin/users                 dr.chen            OK
403     403   GET   /api/admin/users                 nurse.patel        OK
403     403   GET   /api/admin/users                 alice              OK
403     403   GET   /api/admin/users                 bob                OK
401     401   GET   /api/admin/audit                 -                  OK
200     200   GET   /api/admin/audit                 admin              OK
403     403   GET   /api/admin/audit                 dr.chen            OK
403     403   GET   /api/admin/audit                 nurse.patel        OK
403     403   GET   /api/admin/audit                 alice              OK
403     403   GET   /api/admin/audit                 bob                OK
401     401   GET   /api/nothing                     -                  OK
403     403   GET   /api/nothing                     admin              OK
403     403   GET   /api/nothing                     dr.chen            OK
403     403   GET   /api/nothing                     nurse.patel        OK
403     403   GET   /api/nothing                     alice              OK
403     403   GET   /api/nothing                     bob                OK
401     401   GET   /api/public/info                 nurse.patel:wrong  OK
401     401   GET   /api/me                          nurse.patel:wrong  OK
401     401   GET   /api/patients/1/record           nurse.patel:wrong  OK
401     401   GET   /api/admin/audit                 nurse.patel:wrong  OK
401     401   GET   /api/public/info                 mallory:x          OK
401     401   GET   /api/me                          mallory:x          OK
401     401   GET   /api/patients/1/record           mallory:x          OK
401     401   GET   /api/admin/audit                 mallory:x          OK

demo: 80 rows checked, 0 mismatch(es)
```

These results meet the response measures of Section 2:

- **S1:** every request with missing or bad credentials (the 11 anonymous rows to protected URLs and the 8 bad-credential rows) got 401, and none returned protected data.
- **S2:** each of the 72 cells of the access matrix returned the expected status, so there were 0 unauthorized grants. Both patients got 403, never 404, for ids other than their own.

Audit log excerpt

The demo prints the audit log as admin when it finishes. Representative entries from this run, one of each kind of event:


```
{"at":"2026-10-20T23:59:14.512126Z","kind":"AUTHZ_DENIED","username":"anonymous","method":"G
{"at":"2026-10-20T23:59:15.337073Z","kind":"AUTHN_SUCCESS","username":"bob","method":"GET","
{"at":"2026-10-20T23:59:15.338095Z","kind":"AUTHZ_DENIED","username":"bob","method":"GET","p
{"at":"2026-10-20T23:59:17.609182Z","kind":"AUTHZ_DENIED","username":"nurse.patel","method":
{"at":"2026-10-20T23:59:18.579959Z","kind":"AUTHZ_DENIED","username":"anonymous","method":"G
{"at":"2026-10-20T23:59:19.356676Z","kind":"AUTHN_FAILURE","username":"mallory","method":"GE
```


The detail field shows which check refused a request. `record` is the own-record check on `PatientController.record`, a per-actor denial. `ROLE_DOCTOR` is the URL rule's required role, a per-role denial. `denyAll` is default deny, and `authenticated` is an anonymous caller refused by the "any logged-in user" rule. Every authenticated request leaves an `AUTHN_SUCCESS` entry, because under stateless HTTP Basic each request is a new authentication.